

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division:C Batch: 2 Practical No: 3

Roll no: 67 PRN No: 12415134 Name of Student: Surabhi Patel

Subject Name : Deep Learning

Problem Statement : Implement forward propagation and backpropagation using TensorFlow/Keras. Analyze the effect of different learning rates and the number of epochs on model performance

Algorithm :

1. Import the required Python libraries for TensorFlow, Keras, NumPy, Matplotlib, and dataset handling.
2. Load the IMDB dataset from the TensorFlow/Keras datasets library.
3. Separate the dataset into training and testing sets.
4. Preprocess the text data by limiting the vocabulary size and padding all input sequences to a fixed length using `pad_sequences()`.
5. Create a Sequential Neural Network model.
6. Add an Embedding layer to convert words into dense vector representations.
7. Add a Global Average Pooling layer to reduce the sequence dimensions.
8. Add one or more Dense (hidden) layers with the ReLU activation function.
9. Add an output Dense layer with the Sigmoid activation function for binary sentiment classification.
10. Compile the model using the Binary Crossentropy loss function, Adam optimizer, and Accuracy as the evaluation metric.
11. Define different learning rates (e.g., **0.1, 0.01, and 0.001**) for performance comparison.
12. Train the model using the training dataset with the `fit()` method for a fixed number of epochs.
13. During **forward propagation**, pass the input data through all network layers to generate predictions.
14. Compute the loss by comparing the predicted outputs with the actual target labels.
15. Perform **backpropagation** to calculate gradients of the loss with respect to the model parameters.
16. Update the weights and biases using the optimizer and the selected learning rate.

17. Repeat the training process using different learning rates and different numbers of epochs (e.g., **5, 10, and 15 epochs**).
18. Evaluate the trained model using the testing dataset with the `evaluate()` method.
19. Record the testing accuracy and loss for each learning rate and epoch combination.
20. Plot the Training Accuracy and Validation Accuracy curves against the number of epochs.
21. Plot the Training Loss and Validation Loss curves to visualize the learning process.
22. Compare the performance of the model for different learning rates and epoch values.
23. Analyze how the learning rate affects the convergence speed and model accuracy.
24. Analyze how increasing the number of epochs affects the training loss, accuracy, and the possibility of overfitting.
25. Identify the learning rate and epoch combination that provides the best overall model performance.
26. Display the final results and draw conclusions based on the experimental observations.

Code :

```
1. import tensorflow as tf
2. from tensorflow.keras.models import Sequential
3. from tensorflow.keras.layers import Dense
4. from tensorflow.keras.optimizers import Adam
5. import matplotlib.pyplot as plt
6. import pandas as pd
7. from tensorflow.keras.datasets import imdb
8.
9. (X_train, y_train), (X_test, y_test) = imdb.load_data(num_words=10000)
10. x_train=x_train.reshape(-1,784)/255.0
11. x_test=x_test.reshape(-1,784)/255.0
12. def create_model(lr):
13.     m=Sequential([Dense(128,activation='relu',input_shape=(784,)),
14.                   Dense(64,activation='relu'),
15.                   Dense(10,activation='softmax')])
16.     m.compile(optimizer=Adam(learning_rate=lr),
17.               loss='sparse_categorical_crossentropy',
18.               metrics=['accuracy'])
19.     return m
20. learning_rates=[0.1,0.01,0.001]
21. results=[]
22. for lr in learning_rates:
23.     model=create_model(lr)
24.     model.fit(x_train,y_train,epochs=5,batch_size=128,verbose=0)
25.     loss,acc=model.evaluate(x_test,y_test,verbose=0)
26.     results.append([lr,acc])
27. df=pd.DataFrame(results,columns=['Learning Rate','Accuracy'])
```

```

28. print(df)
29. plt.plot(df['Learning Rate'], df['Accuracy'], marker='o')
30. plt.xscale('log'); plt.grid(); plt.show()
31. epochs_list=[5,10,20]
32. res=[]
33. for ep in epochs_list:
34.     model=create_model(0.001)
35.     h=model.fit(x_train,y_train,validation_split=0.2,epochs=ep,batch_size=
        128,verbose=0)
36.     loss,acc=model.evaluate(x_test,y_test,verbose=0)
37.     res.append([ep,acc])
38. edf=pd.DataFrame(res,columns=['Epochs','Accuracy'])
39. print(edf)
40. plt.plot(edf['Epochs'],edf['Accuracy'],marker='o')
41. plt.grid(); plt.show()
42.
43. model=create_model(0.001)
44. history=model.fit(x_train,y_train,validation_split=0.2,epochs=20,batch_size=128)
45. plt.plot(history.history['accuracy']);
    plt.plot(history.history['val_accuracy']); plt.legend(['Train','Val']);
    plt.show()
46. plt.plot(history.history['loss']); plt.plot(history.history['val_loss']);
    plt.legend(['Train','Val']); plt.show()
47. print(model.evaluate(x_test,y_test))

```

Output :



